

MASTER_INTEGRATION_PLAN

HelixAgent Master Integration Plan

Comprehensive plan for porting CLI agent features into HelixAgent Date: 2026-04-04 Based on: Analysis of 50+ CLI agents

Executive Summary

After analyzing **50+ CLI agents** across 5 tiers, we have identified **50+ unique features** to port into HelixAgent. This plan provides a prioritized roadmap for implementation.

Phase 1: Foundation COMPLETE

Implemented Features

- 1.  **Tool System** - 30+ tools from CLI agents
- 2.  **Permission System** - 4-layer architecture
- 3.  **Plan Mode** - Multi-step planning with verification
- 4.  **Team Management** - Agent swarms with consensus
- 5.  **KAIROS Service** - Always-on background assistant
- 6.  **Dream System** - Memory consolidation
- 7.  **SubAgent System** - Full lifecycle management
- 8.  **Comprehensive Tests** - All passing

Status: Complete and committed to all upstreams

Phase 2: Advanced Features (Next 2-4 weeks)

2.1 Repository Mapping (From Aider)

Priority: CRITICAL

Features to Port: - Tree-sitter integration for code parsing - Repository structure analysis - Intelligent file context selection - Language detection (100+ languages)

Implementation:

```
// internal/tools/repomap/
type RepoMap struct {
    RootPath      string
    Languages      map[string]LanguageInfo
    Files          []FileInfo
    Symbols        []Symbol
    Dependencies   []Dependency
}
```

```

type Symbol struct {
    Name      string
    Type      string // function, class, variable
    File      string
    Line      int
    Language  string
    Signature string
}

```

Files to Create: - internal/tools/repomap/repomap.go - internal/tools/repomap/parser.go - internal/tools/repomap/symbols.go

2.2 Sandboxed Execution (From Codex)

Priority: CRITICAL

Features to Port: - Container-based sandboxing - Seatbelt integration (macOS) - Network isolation - Resource limits - Audit logging

Implementation:

```

// internal/tools/sandbox/
type SandboxConfig struct {
    EnableNetwork    bool
    ResourceLimits   ResourceLimits
    WorkingDir       string
    Mounts           []Mount
}

type Sandbox struct {
    Runtime  string // docker, podman, seatbelt
    Config   SandboxConfig
    Container string
}

func (s *Sandbox) Execute(ctx context.Context, cmd Command)
    (*Result, error)

```

Files to Create: - internal/tools/sandbox/sandbox.go - internal/tools/sandbox/docker.go - internal/tools/sandbox/seatbelt.go

2.3 Browser Automation (From Cline/OpenHands)

Priority: HIGH

Features to Port: - Headless browser control - Screenshot capture - DOM interaction - Web navigation - Content extraction

Implementation:

```
// internal/tools/browser/
type BrowserTool struct {
    Headless bool
    Timeout  time.Duration
}

type BrowserAction struct {
    Type      string // navigate, click, type, screenshot
    Target    string
    Value     string
}

func (b *BrowserTool) Execute(ctx context.Context, action
    BrowserAction) (*BrowserResult, error)
```

Files to Create: - internal/tools/browser/browser.go - internal/tools/browser/actions.go

2.4 Edit Block Format (From Aider/Codex)

Priority: HIGH

Features to Port: - Search/replace blocks - Surgical code modifications - Minimal diff generation - Multi-file editing

Implementation:

```
// internal/tools/editblock/
type EditBlock struct {
    FilePath    string
    Search      string
    Replace     string
    Context     int // lines of context
}

type EditBlockResult struct {
    Success     bool
    Blocks      []AppliedBlock
    Failed      []FailedBlock
}

func ApplyEditBlocks(ctx context.Context, blocks []EditBlock)
    (*EditBlockResult, error)
```

Files to Create: - internal/tools/editblock/editblock.go - internal/tools/editblock/parser.go

Phase 3: Enhanced Features (Weeks 4-6)

3.1 Voice Commands (From Aider)

Priority: MEDIUM

Implementation:

```
// internal/agents/voice/  
type VoiceService struct {  
    Recognizer SpeechRecognizer  
    Commands   map[string]VoiceCommand  
}
```

3.2 Auto-Commit Workflow (From Aider/Codex)

Priority: MEDIUM

Implementation:

```
// internal/tools/git/  
func AutoCommit(ctx context.Context, changes []Change) (*Commit, error)  
func GenerateCommitMessage(ctx context.Context, diff string) (string, error)
```

3.3 Evaluation Framework (From OpenHands/SWE-agent)

Priority: MEDIUM

Implementation:

```
// internal/benchmark/  
type Evaluator struct {  
    Benchmarks []Benchmark  
    Metrics    []Metric  
}  
  
type Benchmark interface {  
    Run(ctx context.Context, agent Agent) (*Result, error)  
}
```

3.4 Context Providers (From Continue)

Priority: MEDIUM

Implementation:

```
// internal/context/
type ContextProvider interface {
    Name() string
    Resolve(ctx context.Context, query string) ([]ContextItem,
        error)
}

type FileProvider struct{}
type URLProvider struct{}
type DocsProvider struct{}
```

Phase 4: Integration (Weeks 6-8)

4.1 Multi-Agent Swarm (From Claude Code)

Priority: MEDIUM

Enhance existing Team Management with: - XML-based communication - Shared scratchpad - Color assignment - Role-based agents

4.2 Agent Modes (From Roo Code)

Priority: LOW

```
// internal/agents/modes/
const (
    ModeCode      AgentMode = "code"
    ModeArchitect AgentMode = "architect"
    ModeAsk        AgentMode = "ask"
    ModeDebug      AgentMode = "debug"
)
```

4.3 YOLO Classifier (From Claude Code)

Priority: LOW

ML-based auto-approval system for tool execution.

Implementation Schedule

Week	Feature	Priority	Status
1	Repository Mapping	CRITICAL	🕒
1-2	Sandboxed Execution	CRITICAL	🕒
2	Browser Automation	HIGH	🕒
2-3	Edit Block Format	HIGH	🕒

Week	Feature	Priority	Status
3	Voice Commands	MEDIUM	🕒
4	Auto-Commit	MEDIUM	🕒
5	Evaluation Framework	MEDIUM	🕒
6	Context Providers	MEDIUM	🕒
7-8	Integration & Polish	LOW	🕒

File Structure

```
internal/
├── tools/
│   ├── repomap/           # Repository mapping
│   ├── sandbox/          # Sandboxed execution
│   ├── browser/          # Browser automation
│   ├── editblock/        # Edit block format
│   └── git/               # Enhanced git ops
├── agents/
│   ├── voice/             # Voice commands
│   ├── modes/            # Agent modes
│   └── swarm/             # Multi-agent coordination
├── context/
│   └── providers/         # Context providers
├── benchmark/
│   └── evaluators/       # Evaluation framework
```

Testing Strategy

- 1. **Unit Tests** - Each component
- 2. **Integration Tests** - Component interactions
- 3. **E2E Tests** - Full workflows
- 4. **Benchmark Tests** - Performance evaluation
- 5. **Security Tests** - Sandbox isolation

Success Metrics

- ☐ 90%+ test coverage
- ☐ All critical features implemented
- ☐ Security audit passed
- ☐ Performance benchmarks met
- ☐ Documentation complete

Next Immediate Actions

1. Start **Repository Mapping** implementation
2. Begin **Sandboxed Execution** design
3. Research **Browser Automation** libraries
4. Draft **Edit Block** specification

Status: Ready to begin Phase 2 implementation